

Replay and Reinforce: Reusing One Demonstration to Learn Cross-Object Contact-Rich Manipulation

Anonymous Authors

Abstract—Learning contact-rich manipulation often requires substantial demonstrations to capture task-specific interaction dynamics. This challenge becomes more pronounced in cross-object settings, where demonstrations that succeed on one object may fail on another with different interaction dynamics. Rather than collecting new demonstrations for every object, we investigate whether a single source-task demonstration can provide a reusable prior for online adaptation to a related target object. We present Replay and Reinforce RL (R^2 RL), an off-policy RL framework for cross-object online adaptation from a single source-task demonstration. R^2 RL treats the recorded trajectory as a persistent replay policy rather than merely as replay data. At each step, an ensemble-based uncertainty-aware Q-gate selects between the replay action and the current RL action, allowing the policy to exploit transferable behavior early while departing from the demonstration as target-task experience accumulates. We evaluate R^2 RL on two physical-robot transfer pairs, SnapOpen $\rightarrow$ {SnapOpenNew, OpenBox}, and PegInHole $\rightarrow$ PlugIn, where the source demonstration fails when directly replayed on the new target object. With only two demonstrations, R^2 RL learns five contact-rich tasks and achieves 90–100% success rate across tasks through only 10–30 minutes of real-world training. Among the evaluated real-world sample-efficient RL baselines, R^2 RL is the only method that solves both source tasks and target tasks with different contact dynamics model. Ablation results further suggest that BC regularization helps retain the demonstration prior, and uncertainty-aware gating improves learning efficiency when direct replay fails.

I. INTRODUCTION

Collecting real-world demonstrations for learning contact-rich manipulation is costly: successful skill acquisition often requires extensive experience to capture task-specific forces, compliance, friction, and geometric tolerances [1]. This challenge becomes more pronounced in cross-object settings: even for the same manipulation skill, differences in object stiffness, compliance, friction, or tolerance can cause a trajectory that succeeds on one object to fail on another [2]. One straightforward solution is to collect new expert demonstrations for each object, but this incurs substantial human effort and may need to be repeated whenever the object properties change [3], [4]. In contrast, humans can often learn a manipulation skill from very limited experience and subsequently transfer it across objects. For instance, after seeing how to open one beverage can, a person need not see another demonstration for another new can. The one can-open demonstration provides a useful prior, while online interaction feedback resolves the object-specific dynamics.

Motivated by this observation, this paper asks and attempts to answer the core question: *Can one source-task demonstration provide a useful prior to accelerate online adaptation of the same manipulation skill to a dynamically different*

object? We study this question through online Reinforcement Learning (RL), which provides a natural mechanism for adapting the demonstrated skill using reward feedback collected directly on the target object. Recent sample-efficient RL approaches have incorporated human demonstrations for pretraining, and have shown promising potential for efficient online learning and optimization of real-world contact-rich robotic skills [5]–[7]. However, these methods assume access to multiple successful demonstrations collected directly on the task being learned. We consider a more constrained setting in which only a single trajectory from a related source object is available, and direct replay of this trajectory may fail on the target task. How to effectively exploit such a weak demonstration prior within off-policy RL remains insufficiently studied. To enable cross-object adaptation from only one source-task demonstration, we propose Replay and Reinforce RL (R^2 RL), as illustrated in Fig. 1.

The key premise of this work is that a demonstration can remain informative even when it is not directly executable on the target object. To the best of our knowledge, R^2 RL is the first real-world online RL framework to adapt across objects from a single source demonstration without requiring target-object demonstrations. More specifically, our contributions are threefold: (1) we introduce R^2 RL, which treats the single demonstration as a replay policy rather than merely a one-shot initialization signal, and introduces uncertainty-aware Q-gating. This allows the demonstration to remain useful not only for optimizing the source task, but also for guiding cross-object adaptation even when direct replay fails on the target task. (2) we evaluate R^2 RL on two physical-robot transfer pairs, SnapOpen $\rightarrow$ {SnapOpenNew, OpenBox} and PegInHole $\rightarrow$ PlugIn. A single demonstration is collected for each source task and is used both to optimize the source task and to adapt to its corresponding target task, yielding five learned policies from only two demonstrations. The experimental results show that, R^2 RL achieves 90–100% success rates through only 10–30 minutes of real-world training. Compared with state-of-the-art sample-efficient RL, R^2 RL is the only method that solves cross-object adaptation when the available demonstration originates from a different object. (3) Through systematic ablations, we show that Q-gating supports more efficient cross-object adaptation than residual RL under the same single-demonstration budget, while BC regularization helps retain the demonstration prior and uncertainty-aware Q-gating further improves learning efficiency, particularly when the replay of source-task demonstration fails on target tasks.

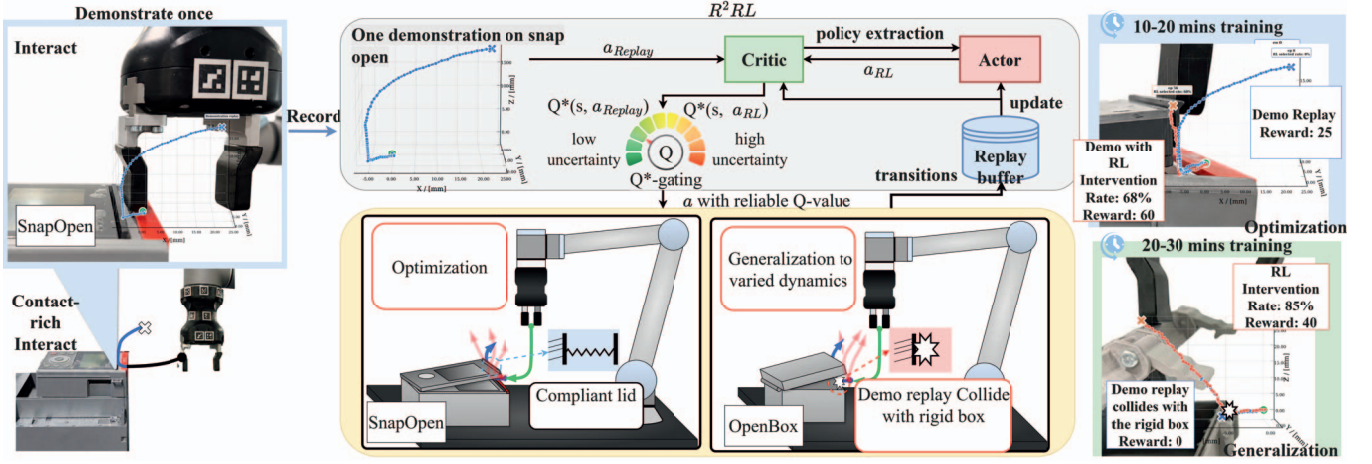


Fig. 1: Overview of R^2RL . A single source-task demonstration is retained as a replay policy. At each step, the replay policy and the learned RL policy propose candidate actions, and an uncertainty-aware Q-gate selects the action with the more reliable value estimate. The same demonstration is used both to optimize the source task and to adapt to a target object where direct replay fails because of different interaction dynamics.

II. RELATED WORK

A. Learning from a Single Demonstration

Targeting the high cost of data collection in robotics, one-shot learning approaches use a single demonstration to generalize manipulation skills to new settings [3], [8]–[12]. Many methods focus on transferring the demonstrated trajectory across changes in object pose, appearance, or geometry. YODO transfers trajectories across object instances through category-level representations [3], while DINOBot uses DINO features for visual retrieval and alignment before trajectory replay [10]. IMOP similarly transfers actions from a single demonstration by matching invariant regions between the demonstrated and target scenes [9]. Self-Replay learns to reach the demonstrated bottleneck pose under new configurations and subsequently replays the recorded interaction actions, assuming that accurate alignment makes the demonstrated interaction directly reusable [12]. YOTO extends one-shot learning to long-horizon bimanual manipulation from a single human video [11]. However, these methods mainly adapt the demonstrated behavior through alignment, retargeting, or structured task representations, rather than reusing it to learn a policy that adapts across objects with different contact dynamics.

A related line of work also reuses a demonstration to accelerate new skill learning rather than directly replaying it. RoboCLIP derives reward signals from the demonstration using a pretrained video-and-language model for online RL across varied tasks [13]. One can also reuse the demonstration at the primitive level: probabilistic movement primitives can be transferred to initialize the policy-search distribution for a new task [14], while libraries of motion primitives can be retrieved and geometrically transformed to satisfy new task constraints [15]. These methods show how one demonstration can provide useful structure for learning, but do not specifically consider cross-object online adaptation where the demonstration may fail on the new target object.

B. RL for Cross-Object Adaptation

For cross-object adaptation, residual RL adapts a nominal controller by learning corrective actions from interaction, while rLfD extends this idea to demonstrated skills and shows adaptation across variations in geometry and friction [2], [16]. These methods assume a reusable nominal policy or trajectories and learn residual corrections for the new conditions. In particular, rLfD constructs a DMP from one demonstration and learns an additive task-space correction on the new condition. Unlike residual learning, which adapts behavior through additive corrections to a fixed base action, R^2RL treats the source action as a proposal and can be completely replaced with a learned action when the critic identifies it as unsuitable for the target dynamics.

More closely related, CRSfD studies RL with *mismatched demonstrations*, where demonstrations collected on a source task are transferred to target tasks with different geometries or rewards through conservative reward shaping [17]. MoDem-V2 similarly addresses adaptation in contact-rich manipulation by initializing from a Behavior-Cloned (BC) policy and gradually transferring control to model-based RL [18]. In contrast, R^2RL targets cross-object adaptation from only a single source trajectory. Rather than converting the demonstration into reward shaping or first learning a source policy, R^2RL retains the trajectory as an action prior and uses target-task value and uncertainty to decide online when to reuse or override demonstrated actions.

C. Demonstration-Guided Reinforcement Learning

Demonstrations are widely used to improve exploration and sample efficiency for online RL, through replay, imitation losses, or offline-to-online initialization [5]–[7], [19]–[22]. In real-world settings, RLPD serves as the RL backbone for SERL and HIL-SERL, enabling sample-efficient learning of contact-rich manipulation skills [6], [7]. SERL contributes a practical software and algorithmic framework for sample-efficient real-world RL, while HIL-SERL extends this setting with human interventions to improve learning efficiency

and task success. IBRL instead learns an imitation policy from demonstrations and uses a novel Q-gating mechanism to select between the frozen imitation policy and RL actions during exploration and value learning [5]. However, these existing methods assume access to multiple successful demonstrations of the target task. In contrast, R^2 RL requires only a single trajectory from a related source object, which may fail when directly executed on the target, and retains it as a persistent action prior to accelerate online RL adaptation.

III. PROBLEM SETTING

We formulate each task as a Markov Decision Process (MDP). At each time step t , the agent observes a state $s_t \in \mathcal{S}$ and executes an action $a_t \in \mathcal{A}$ according to the policy $\pi_\theta(s_t)$. It then receives a reward $r_t = \mathcal{R}(s_t, a_t)$ and a future state s_{t+1} based on dynamic transition: $s_{t+1} \sim \mathcal{P}(s_{t+1} | s_t, a_t)$. The objective is to learn a policy π_θ that maximizes the expected discounted return $J(\theta) = \mathbb{E}[\sum_{t=0}^T \gamma^t r_t]$ during learning, where the $\gamma \in [0, 1)$ is the discount factor and T is the maximum episode length.

We consider a *source-to-target pair*, where the source and target tasks share the same state and action space but may have different transition dynamics $\mathcal{P}$ and rewards $\mathcal{R}$. In both tasks, the state contains robot motion and wrench measurements, $s_t = [m_t, w_t]$, where m_t denotes the robot motion state and w_t denotes the measured force and torque. Before learning begins, one successful demonstration from the source task is available and is reused for both source-task optimization and target-task adaptation. We restrict attention to source-target pairs for which the demonstrated action sequence provides a meaningful coarse motion prior after both tasks are expressed in a common Cartesian frame. Specifically, the source and target tasks should share the same coarse task-progress direction, as determined from the task geometry or goal specification. We additionally assume approximate temporal compatibility, such that the demonstrated trajectory reaches the corresponding interaction or contact phase at approximately the same stage of the episode on the source and target tasks, i.e.,

$$\left| \frac{t_c^{Source}}{T_{Source}} - \frac{t_c^{Target}}{T_{Target}} \right| \leq \epsilon_t, t_c^i = \min \{t : \|w_t^i\|_2 > \epsilon_F\} \quad (1)$$

where t_c^i denotes the first contact time step, $i \in \{Source, Target\}$, *Source*, *Target* are short for Source task and Target task respectively, and ϵ_t is the allowed temporal mismatch. The small threshold ϵ_F prevents the force/torque sensor noises from being considered as contact. These assumptions do not imply that the demonstrated actions are directly successfully executable on the target object; differences in contact dynamics, compliance, friction, or geometric tolerance may still cause direct replay to fail.

The central challenge is therefore, given a single demonstration, how to effectively incorporate it into training such that: 1) it is not forgotten as the growing training dataset dominates the replay buffer, 2) the RL policy does not overfit to it, hindering exploration; and most importantly, 3) it can

remain useful for accelerating cross-object adaptation even when the source task demonstration fails the target task.

IV. PRELIMINARIES

We adopt an off-policy actor-critic RL, which includes a stochastic policy network π_θ , critic networks Q_ϕ , a replay buffer $\mathcal{B}$ and a demonstration buffer $\mathcal{D}$. R^2 RL combines Q-gating based action selection with an in-sample critic update target. The Q-gating mechanism follows IBRL [5], which selects between actions from an imitation policy trained from multiple successful demonstrations and an RL policy using a learned critic:

$$a(s) = \operatorname{argmax}_{\pi \in \{\pi^{IL}, \pi^{RL}\}} Q_{\phi'}(s, \pi(s)). \quad (2)$$

This preserves a demonstrated behavior during online learning, but it requires an imitation policy and ordinary Q-gating can be unreliable before either proposal has produced reward.

For critic learning, R^2 RL adopts the update objective of Implicit Q-Learning (IQL) to prevent Q-value overestimation on out-of-distribution (OOD) action [23]. Although IQL is proposed for Offline RL, it has also been proved a good method for online fine-tuning. Compared to standard Bellman backup:

$$\mathcal{L}(\phi) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{B}} \left[(Q_\phi(s, a) - (r + \gamma Q_\phi(s', \pi_\theta(s'))))^2 \right], \quad (3)$$

where $s', \pi_\theta(s')$ denote the future rollouts. IQL does not require the policy predictions during critic update. A Value function $V_\psi(s)$ is used to select better actions implicitly for each state s and estimate bootstrapped next-state value. The critic is then updated using

$$\mathcal{L}_{IQL}(\phi) = \mathbb{E}_{(s_t, a_t, s_{t+1}, r_t) \sim \mathcal{B}} \left[(r_t + \gamma V_\psi(s_{t+1}) - Q_\phi(s_t, a_t))^2 \right] \quad (4)$$

IQL uses exponential regression to learn V_ψ :

$$\mathcal{L}_V(\psi) = \mathbb{E}_{(s_t, a_t) \sim \mathcal{B}} [\mathcal{L}_\tau(Q_\phi(s_t, a_t) - V_\psi(s_t))] \quad (5)$$

Where $\mathcal{L}_\tau(u) = |\tau - \mathbf{1}(u < 0)| \cdot u^2$.

V. METHODOLOGY

A. A Demonstration as a Replay Policy

One demonstration is not enough to serve as a conventional offline dataset for sampling or pretrain a policy, thus, we propose to view the single demonstration τ^D as a time-indexed replay policy a_t^D , where t denotes the control timestep within an episode. To match the task horizon, the demonstration is resampled to the episode length T , yielding $\tau^D = (s_t^D, a_t^D)_{t=0}^T$. At the beginning of each episode, the replay index is reset to zero. At t , the replay policy proposes the corresponding demonstrated action a_t^D , while a randomly initialized stochastic actor proposes $a_t^{RL} \sim \pi_\theta(\cdot | s_t)$.

Following the Q-gating mechanism introduced in Eq. (2), R^2 RL evaluates both candidate actions at the current state s_t and executes the one with the larger estimated return:

$$a_t = \operatorname{argmax}_{a \in \{a_t^D, a_t^{RL}\}} Q_\phi^*(s_t, a). \quad (6)$$

Although a_t^D is indexed by time rather than state-conditioned, its execution is state dependent because the critic evaluates it at the observed state s_t . Therefore, temporal indexing determines which demonstrated action is proposed, whereas the learned critic determines whether that action remains appropriate under the current state. Consequently, as the critic becomes more informative, this mechanism can retain useful portions of the source behavior while allowing the RL policy to replace actions that are unsuitable under the target-object dynamics.

B. Uncertainty-Aware Q-Gating

Standard Q-gating selects more reliable actions in regions well supported by previously observed data [24], but with only one demonstration, early Q-estimates can be poorly calibrated and giving overestimation to OOD state-action pair. This issue is especially problematic during cross-object adaptation, where direct replay of the source demonstration and early policy rollouts may fail and both receive zero reward. Here, the Q_ϕ alone is no longer an effective criterion for selecting between a_t^D and a_t^{RL} , and an overestimated OOD policy action may prematurely displace the demonstration prior. To account for the reliability of each value estimate, R^2RL maintains an ensemble of K critics [25]. For a candidate action a , the ensemble mean and its variance-derived epistemic uncertainty are

$$\mu_Q(s, a) = \frac{1}{K} \sum_{k=1}^K Q_k(s, a), \quad \sigma_Q(s, a) = \sqrt{\text{Var}_k[Q_k(s, a)]}. \quad (7)$$

We define the uncertainty-aware Q-gating as follows:

$$Q^*(s, a) = \mu_Q(s, a) + \exp[-\lambda \sigma_Q(s, a)]. \quad (8)$$

Where $\lambda > 0$ controls the sensitivity of the gate to critic disagreement: a larger λ gives a larger penalty to critic disagreement. The exponential term provides a bounded confidence bonus and adds sensitivity to critic disagreement. Since adding an action-independent constant does not change the gating decision, Eq. (8) is equivalently written as $Q^*(s, a) = \mu_Q(s, a) - (1 - \exp[-\lambda \sigma_Q(s, a)])$. Therefore, the proposed score can be interpreted as a bounded lower-confidence surrogate, where $1 - \exp[-\lambda \sigma_Q(s, a)] \in [0, 1]$ acts as an uncertainty penalty. For small disagreement, $1 - \exp(-\lambda \sigma) \approx \lambda \sigma_Q$, locally recovering the conventional linear uncertainty penalty. Compared with the direct linear penalty $-\lambda \sigma_Q$, the marginal effect of the exponential penalty decreases with increasing disagreement and eventually saturates. Consequently, critic disagreement can guide early gating decisions without unboundedly suppressing an action whose estimated-return later becomes sufficiently large.

C. Policy and Critic Update

R^2RL includes two memory buffers: the demonstration buffer τ^D , which includes the demonstration and supplies replay proposals. An initially empty replay buffer $\mathcal{B}$ stores complete transitions collected during online interactions. Because the critics are initially uninformative, R^2RL only

executes the replay proposal throughout the first episode. The resulting transitions are stored in $\mathcal{B}$, allowing the critics to observe the actual expected return of the demonstrated actions under the target dynamics. The policy network is updated with a behavior-regularized actor objective [26]:

$$\mathcal{L}_\pi(\theta) = -\mathbb{E}_{(s,a) \sim \mathcal{B}} [Q^*(s, \pi_\theta(s)) + \kappa \log \pi_\theta(a | s)]. \quad (9)$$

The likelihood term keeps RL policy update closer to the behavior policy and therefore makes real-world online learning more stable. We use an adaptive gain $\kappa = |\mathbb{E}[\sigma_\pi]|$, increasing regularization when the stochastic actor is uncertain. Because online transitions rapidly dominate $\mathcal{B}$, this term regularizes toward the agent's improving behavior rather than permanently cloning the single demonstration.

For the critic networks update, we adopt IQL, introduced in the Preliminaries. Comparing to the standard Bellman backup, the IQL update objective (Eq.(4)) does not require computing RL policy outputs for future rollouts in the Bellman target, reducing reliance on potentially poorly estimated OOD actions during early online adaptation.

VI. EXPERIMENTAL EVALUATION

We design the experiments and the evaluation around three questions: 1) can a single source trajectory accelerate online learning on both its source task and a related target object, even when direct replay fails on the target? 2) how does R^2RL compare with sample-efficient online RL baselines under the same real-robot budget? and 3) which elements of R^2RL are responsible for successful adaptation?

Experiments use a 6-DoF Universal Robot UR5e with its inbuilt force/torque sensor and equipped Robotiq Hand-E parallel-jaw gripper; control is handled with UR RTDE [27]. All experiments were trained directly on real-world setups without the need for sim-to-real transfer, and each training was repeated 3 times to provide empirical results. Implementation details on tasks and training configurations are presented as well.

A. Real-World Tasks Implementations and Protocol

Five contact-rich manipulation tasks used for evaluation are illustrated in Fig.2. In all evaluated pairs, the compatibility conditions are satisfied by construction of the task coordinate frame. All tasks use Cartesian velocity control of the Tool Center Point (TCP) at 20 Hz, with a maximum episode length of 100 steps. The actions for all tasks are $[\dot{x}, \dot{y}, \dot{z}, \dot{R}_x, \dot{R}_y, \dot{R}_z]$. For safe exploration during real-world training, the commanded velocities are bounded by $a_{\max} = [0.020, 0.010, 0.014, 0.20, 0.20, 0.20]$ and $a_{\min} = [-0.020, -0.010, -0.006, -0.20, -0.20, -0.20]$ for the first task pair, and $a_{\max} = -a_{\min} = [0.020, 0.020, 0.020, 0.20, 0.20, 0.20]$ for the second, where the translational and rotational units are expressed in ms^{-1} and rads^{-1} . The common observation consists of the TCP pose $[x, y, z, R_x, R_y, R_z]$, the measured wrench $[F_x, F_y, F_z, \tau_x, \tau_y, \tau_z]$, and the previous actions, resulting in an 18-dimensional observation. SnapOpen, SnapOpenNew and OpenBox use progress reward only when the contact

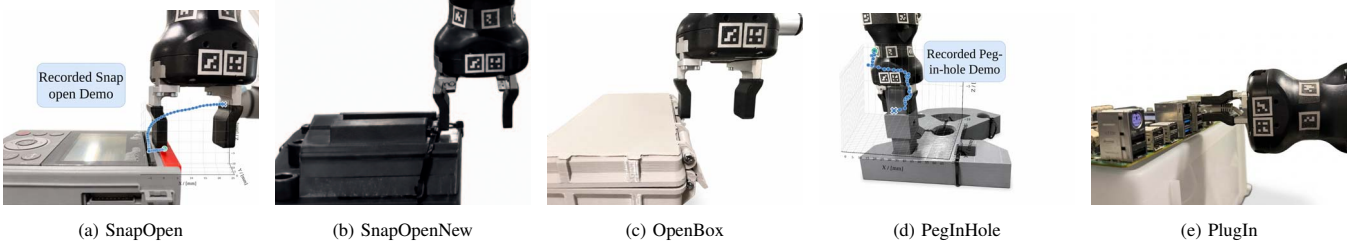


Fig. 2: Overview of two real-world source-to-target pairs for experiments. The first source-to-target pair includes the source task: SnapOpen, and the target tasks: OpenBox and SnapOpenNew. The second source-to-target pair includes the source task: PegInHole, and the target task: PlugIn. Only for the source tasks, SnapOpen and PegInHole, we collected one successful teleoperated demonstration per task at 50 HZ, and linearly interpolate each trajectory over normalized time to match the episode horizon $T = 100$. The demonstration is used both to optimize the source task and to adapt to target tasks. a) SnapOpen, the task requires the robot to maintain compliant contact while disengaging a snap-fit latch; b) SnapOpenNew, SnapOpen task with a new object, whose complaint lid has different friction properties. Directly replaying the source demonstration causes the robot to lose contact with the lid during the lifting motion. c) OpenBox, the task requires the robot to open and lift a rigid box lid. The rigid box lid differs in both compliance and friction from the snap-fit mechanism in the source task, such that directly replaying the source demonstration leads to collision with the lid and consequently fails to complete the task. Nevertheless, the source trajectory still provides a coarse motion prior that brings exploration toward the relevant interaction region. d) PegInHole, the task optimizes insertion performance under randomly introduced initial offsets. e) PlugIn, the USB insertion task requires substantially tighter positional tolerance than the PegInHole task, making direct transfer of the demonstrated behavior insufficient and requiring further online adaptation.

condition is satisfied: $r = 1 - d/h$, $50 N < A_x < 2000 N \wedge F_x > 3 N$; Where $d = \min(h, |z_{\text{tar}} - z_{\text{cur}}|)$, z_{cur} is the current TCP position, $z_{\text{tar}} = z_{\text{init}} + h$, h is the reference height for lifting the control panel. Episodes reset when the terminated or truncated condition is met:

$$R = \begin{cases} 1, & \text{if } t > T \vee |F_x| > 50 N \vee \max(|F_y|, |F_z|) > 30 N \\ 0, & \text{otherwise} \end{cases} \quad (10)$$

where R is short for reset. PegInHole and PlugIn receive unit reward only when the insertion depth passes the task threshold: $r = 1$, $z < z_{\text{hole surface}} - 0.05$. Episodes reset under the same conditions as in Eq. (10), except that the force thresholds are replaced by: $|F_z| > 50 N \vee \max(|F_y|, |F_x|) > 30 N$. Initial TCP position is randomized within ± 2 mm for SnapOpen, OpenBox, and PlugIn, and within ± 5 mm for PegInHole.

B. Cross-Object Online Adaptation

To examine the effectiveness of cross-object adaptation, we focus on the performance of the first experimented source-to-target pair. Fig. 3 compares the normal contact force F_x under demonstration replay and after R^2 RL training, with the x -axis aligned with the dominant contact normal. Under replay, the three objects exhibit distinct interaction responses: SnapOpen maintains contact and completes the demonstrated motion, whereas OpenBox shows a rapid force buildup followed by early termination. SnapOpenNew initially exhibits a force profile similar to SnapOpen, but loses contact near step 64. We can interpret these responses using a simplified contact model, $F_x^{(j)} \simeq m_j \ddot{\delta}_j + c_j \dot{\delta}_j + k_j \delta_j$, $j \in \{\text{source}, \text{SnapOpenNew}, \text{OpenBox}\}$ where δ is the effective normal deformation and m , c , and k denote apparent interaction inertia, damping, and stiffness. The model is used only for qualitative interpretation. Because the lid of OpenBox is more rigid than SnapOpen, $k_{\text{OpenBox}} > k_{\text{source}}$. For comparable replay-induced deformation $\delta^D(t)$, a larger stiffness yields faster force buildup, which is consistent with the OpenBox replay trace. After R^2 RL training, the

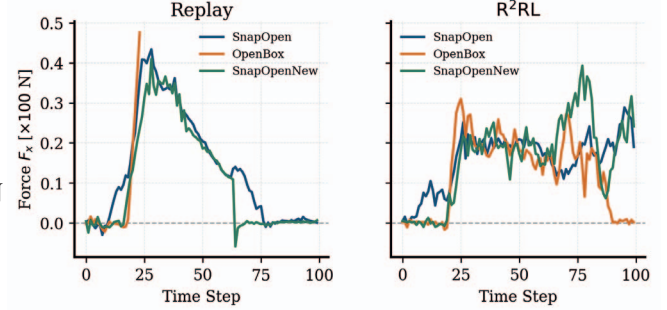


Fig. 3: Normal contact force F_x under demonstration replay and after R^2 RL training for the first source-to-target pair. Replay fails through excessive force buildup on OpenBox and premature contact loss on SnapOpenNew, whereas R^2 RL adapts the interaction to sustain contact.

learned policy regulates F_x below the force limit while maintaining continuous contact with the lid until the box is open. SnapOpenNew has similar compliance, $k_{\text{new}} \approx k_{\text{snap}}$, but different friction. Its tangential contact satisfies $|F_t| \leq \mu_j F_x$, so a change in μ_j alters the sticking–sliding condition and can cause the observed loss of contact near step 64. After training, R^2 RL maintains a stable normal force and sustained contact throughout the lifting phase, indicating that the learned policy adapts the interaction to the changed frictional condition. Thus, these observations and analysis indicate that R^2 RL learned to adapt different contact dynamics using the same source demonstration.

C. Comparisons on Sample-efficient RL

To validate the learning efficiency and adaptation ability using one demonstration and answer question 1 and 2, we benchmark against state-of-the-art off-policy RL algorithms designed for efficient online learning in real-world robotic settings: RLPD [22] and IBRL [5]. The online training performance is shown in Fig.4, both source-to-target task pairs are evaluated. For the first pair, we restrict the target-task comparison to OpenBox, which is the more challenging adaptation target. We evaluate RLPD and IBRL under one

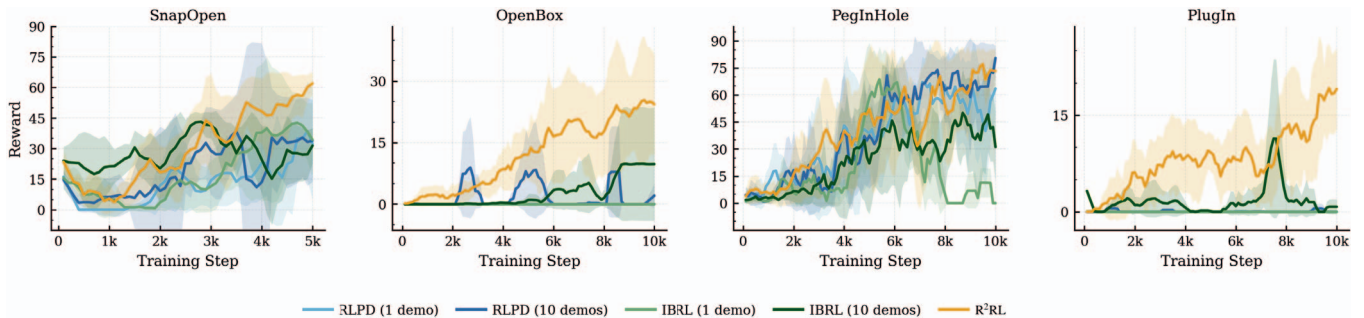


Fig. 4: Comparisons of cumulative rewards during online learning. All methods use learning rate 10^{-4} , batch size 64, discount 0.99, replay capacity 10^4 , update-to-data ratio one, and five critics. R^2 RL uses IQL expectile $\tau = 0.7$ and selects λ as a constant 10 for all experiments. All actor, critic, and value networks are MLPs with hidden dimensions 256–128–64 and ReLU activations. Networks are optimized with Adam using learning rate 10^{-4} . We use 5 independently initialized critic networks for updating critic ensembles.

TABLE I: Policy success rate over 10 real-world evaluation trials. “1/10” denotes the number of source demonstrations supplied during training; no target-task demonstration is supplied for OpenBox or PlugIn. Note: for SnapOpen task, the task is considered success if the compliant lid is snapped and the force limitation is not violated.

Method	SnapOpen	OpenBox	PegInHole	PlugIn
Replay (1 demo)	100	0	20	0
RLPD (1/10 demos)	90/ 100	0/0	60/ 100	0/0
IBRL (1/10 demos)	70/40	0/50	10/90	0/10
R^2 RL (1 demo)	100	100	90	100

and ten source-demonstration settings, while R^2 RL always uses one source demonstration. In all settings, no method receives target-task demonstrations. The one demonstration comparisons use exactly the same successful source trajectory as R^2 RL; the ten demonstrations setting evaluate whether additional source-only demonstrations are sufficient to overcome the source–target mismatch, while also testing whether a single demonstration is sufficient to accelerate learning. For target-task training with RLPD, we execute the recorded source actions on the target object and use the resulting target-domain transitions for initialization. This is to align the transitions under different object dynamics with the same demonstration budget. For IBRL, we use two implementations according to the demonstration budget. In the one-demonstration setting, the single trajectory is represented as the same time-indexed replay proposal used by R^2 RL. In the ten-demonstration setting, IBRL follows its standard formulation: a BC policy is trained from the ten source-task demonstrations and subsequently used as the imitation-policy candidate for Q-gating.

The success rates for learned policy across four tasks are shown in Tab.I. Overall, among the evaluated methods and under the source-demonstration-only protocol, R^2 RL is the only method that obtains high final success on all four tasks, achieving a near-100% success rates with just a single demonstration and 10–30 minutes of real-world training. In the SnapOpen and PegInHole tasks, where demonstrations are provided, RLPD and IBRL with 10 demonstrations can reach similar performance, but their training is less stable than R^2 RL. In OpenBox and PlugIn tasks, where the demonstrations are from source-tasks, both RLPD and

IBRL fail to learn the target tasks, even with 10 source-demonstrations. In contrast, R^2 RL learns stably using only one source-demonstration.

D. Ablation experiments

The ablation experiments are conducted on SnapOpen and OpenBox, representing a source task with rewarded replay and a target task on which direct replay fails, respectively. First, we compare the proposed Q-gating with residual RL to justify viewing replay actions as selectable proposals rather than a base policy. Then, to understand how each component of R^2 RL supports learning from a single demonstration, particularly when direct replay fails, we ablate (1) data incorporation with standard Q-gating Q : as Eq.(2), and Q^* ; (2) policy extraction with and without BC regularization $BC\ reg.$. The results are present in Fig.6a and 6b. Last, to evaluate that IQL is the most effective critic update objective for the proposed data incorporation method, we compared different critic functions, including Bellman backup, IQL, and CQL [28]. The results are present in Fig.6c and 6d.

a) *Q-gating versus Residual RL*: Residual RL provides the closest alternative to Q-gating for retaining demonstrated behavior during online adaptation [2], [16]. We therefore compare the two mechanisms under the same one-demonstration budget to study how the source behavior should be incorporated into learning. For Residual RL, the demonstrated action is treated as a persistent base command and the learned policy produces an additive correction, $a_t = a_t^D + a_t^{RL}$, with a_t^{RL} . We evaluate both standard Residual RL, where a_t^{RL} is learned using TD3, and a regularized variant using the same actor and critic objectives as R^2 RL. Figure 5 reports the cumulative rewards convergence and policy intervention rates during training. R^2 RL’s policy intervention rate is the same as later reported RL selection rates: the fraction of selected RL policy steps in one episode. While the residual RL’s intervention rate is the a_t^{RL} magnitude divided by the sum of the base- and residual-action magnitudes, measuring the relative correction strength. On SnapOpen, both residual variants apply stable corrections but plateau below R^2 RL, whose increasing RL-selection rate coincides with higher reward. The difference is clearer on OpenBox: despite persistent nonzero residual

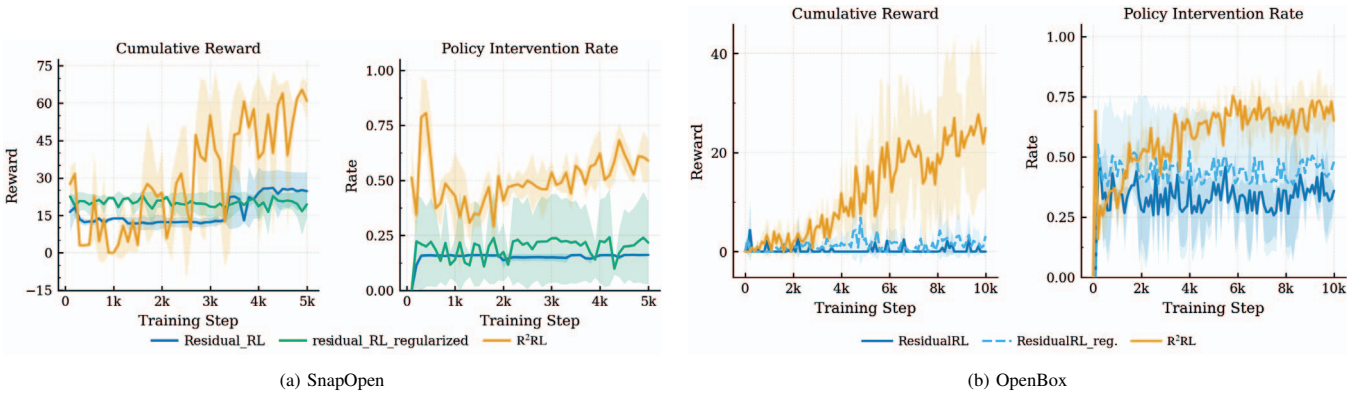


Fig. 5: Comparison between R^2 RL and Residual RL with one source demonstration. Here, the residual RL with reg. is the regularized variant using the same actor and critic objectives as R^2 RL.

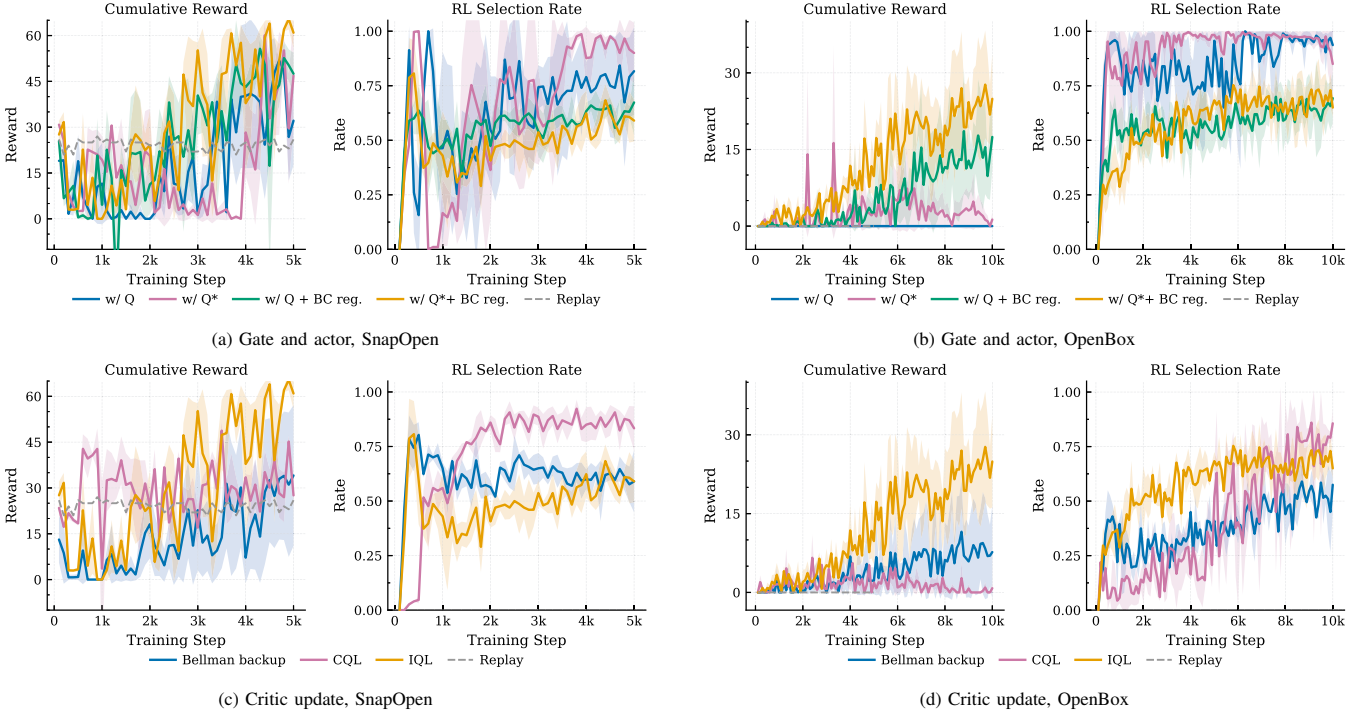


Fig. 6: R^2 RL ablations. Top: standard versus uncertainty-aware Q-gating, with and without BC regularization. Bottom: Bellman, CQL, and IQL critic updates. Reward measures task progress; RL selection rate is the fraction of steps for which the gate executes the learned rather than replay action.

corrections, both residual methods remain near zero reward, whereas R^2 RL progressively replaces replay actions and improves. This suggests that under the evaluated training time, when the demonstration is only a coarse prior, additive correction may be insufficient to escape an incompatible contact strategy, while Q-gating can adapt to the new object faster by directly replacing unsuitable demonstrated actions with learned behaviors.

b) How R^2 RL works when replay fails: On SnapOpen, the demonstration already receives reward, so the ensemble mean dominates the agreement bonus and Q^* provides only a modest improvement (Fig. 6a). OpenBox provides a more revealing case because direct replay of the source demonstration receives zero reward. As shown in Fig. 6b, without BC regularization, both standard and uncertainty-

aware Q-gating drive the RL selection rate toward 1, indicating that the RL policy rapidly takes over from the source demonstration. Although uncertainty-aware Q-gating obtains rewards during training, its performance remains unstable and eventually fails the task. In contrast, adding a BC regularization maintains substantial replay usage throughout training and both variants achieve increasing rewards. This suggests that the source demonstration can still provide a useful exploration prior even when direct replay fails under the target dynamics. Combining BC regularization and uncertainty-aware Q-gating achieves earlier and higher rewards despite broadly similar RL selection rates. The similar selection rates suggest that the improvement comes from Q^* selecting more useful actions, helping the agent use the demonstration prior more effectively to find reward during

early exploration. This is consistent with the Q^* design: using critic disagreement to reduce the influence of early unreliable Q -estimates. Together, these results suggest that BC regularization helps retain the source demonstration as an exploration prior, while uncertainty-aware Q -gating helps use this prior more effectively to guide early exploration during cross-object adaptation.

VII. CONCLUSION AND LIMITATION

The main takeaway of this work is that a demonstration can remain useful for online RL even when it is not directly executable on the target object, which greatly improved the data efficiency. We formulated this setting as one-demonstration cross-object online adaptation and introduced R^2 RL. R^2 RL retains the source trajectory as a persistent replay proposal and uses uncertainty-aware value gating to decide when demonstrated actions should be reused and when they should be replaced by learned actions. Across two contact-rich source–target pairs, two source demonstrations are reused to train 5 separate policies, including 3 target tasks for which direct replay fails. The results therefore demonstrate a successful online cross-object adaptation. The ablations further show that when direct replay fails, BC regularization helps retain the demonstration prior, while uncertainty-aware Q -gating helps use this prior more effectively. Together with in-sample critic learning, these components support stable and efficient real-world policy learning with only one demonstration. One limitation in this work is that each source demonstration is only evaluated on one or two target tasks. The extent to which more complex or semantically different target tasks can be adapted using one-source demonstration remains unexplored. Another limitation in the experiment tasks is that all evaluated tasks are short-horizon; the performance of R^2 RL on long-horizon tasks is still unexplored. For future work, long-horizon tasks will be evaluated.

REFERENCES

- [1] T. Tsuji, Y. Kato, G. Solak, H. Zhang, T. Petrić, F. Nori, and A. Ajoudani, “A survey on imitation learning for contact-rich tasks in robotics,” *The International Journal of Robotics Research*, p. 02783649261417694, 2025.
- [2] T. Davchev, K. S. Luck, M. Burke, F. Meier, S. Schaal, and S. Ramamoorthy, “Residual learning from demonstration: Adapting dmps for contact-rich manipulation,” *IEEE Robot. Autom. Lett.*, vol. 7, no. 2, pp. 4488–4495, 2022.
- [3] B. Wen, W. Lian, K. Bekris, and S. Schaal, “You only demonstrate once: Category-level manipulation from single visual demonstration,” in *Proceedings of Robotics: Science and Systems*, 2022.
- [4] M. Y. Aoyama, J. Moura, N. Saito, and S. Vijayakumar, “Few-shot learning of force-based motions from demonstration through pre-training of haptic representation,” in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 12 839–12 845.
- [5] H. Hu, S. Mirchandani, and D. Sadigh, “Imitation bootstrapped reinforcement learning,” *arXiv preprint arXiv:2311.02198*, 2023.
- [6] J. Luo, Z. Hu, C. Xu, Y. L. Tan, J. Berg, A. Sharma, S. Schaal, C. Finn, A. Gupta, and S. Levine, “Serl: A software suite for sample-efficient robotic reinforcement learning,” in *Proc. IEEE Int. Conf. Robot. Autom.*, 2024, pp. 16 961–16 969.
- [7] J. Luo, C. Xu, J. Wu, and S. Levine, “Precise and dexterous robotic manipulation via human-in-the-loop reinforcement learning,” *Science Robotics*, vol. 10, no. 105, p. eads5033, 2025.
- [8] P. Vitiello, K. Dreczkowski, and E. Johns, “One-shot imitation learning: A pose estimation perspective,” in *Proceedings of the 7th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 229. PMLR, 2023, pp. 943–970.
- [9] X. Zhang and A. Boularias, “One-shot imitation learning with invariance matching for robotic manipulation,” in *Proceedings of Robotics: Science and Systems*, 2024.
- [10] N. Di Palo and E. Johns, “DINOBot: Robot manipulation via retrieval and alignment with vision foundation models,” in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 2798–2805.
- [11] H. Zhou, R. Wang, Y. Tai, Y. Deng, G. Liu, and K. Jia, “You only teach once: Learn one-shot bimanual robotic manipulation from video demonstrations,” in *Proceedings of Robotics: Science and Systems*, 2025.
- [12] N. Di Palo and E. Johns, “Learning multi-stage tasks with one demonstration via self-replay,” in *Proceedings of the 5th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 164. PMLR, 2022, pp. 1180–1189.
- [13] S. A. Sontakke, J. Zhang, S. M. R. Arnold, K. Pertsch, E. Bıyık, D. Sadigh, C. Finn, and L. Itti, “RoboCLIP: One demonstration is enough to learn robot policies,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 55 681–55 693.
- [14] S. Stark, J. Peters, and E. Rueckert, “Experience reuse with probabilistic movement primitives,” in *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2019, pp. 1210–1217.
- [15] B. Hertel, N. Tran, M. Elkoudi, and R. Azadeh, “A framework for learning and reusing robotic skills,” in *2024 21st International Conference on Ubiquitous Robots (UR)*. IEEE, 2024, pp. 801–804.
- [16] T. Johannink, S. Bahl, A. Nair, J. Luo, A. Kumar, M. Loskyll, J. A. Ojea, E. Solowjow, and S. Levine, “Residual reinforcement learning for robot control,” in *2019 International Conference on Robotics and Automation (ICRA)*. IEEE, 2019, pp. 6023–6029.
- [17] Y. Guo, J. Gao, Z. Wu, C. Shi, and J. Chen, “Reinforcement learning with demonstrations from mismatched task under sparse reward,” in *Proceedings of the 6th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 205. PMLR, 2023, pp. 1146–1156.
- [18] P. Lancaster, N. Hansen, A. Rajeswaran, and V. Kumar, “MoDem-V2: Visuo-motor world models for real-world robot manipulation,” in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024.
- [19] M. Vecerik, T. Hester, J. Scholz, F. Wang, O. Pietquin, B. Piot, N. Heess, T. Rothörl, T. Lampe, and M. Riedmiller, “Leveraging demonstrations for deep reinforcement learning on robotics problems with sparse rewards,” *arXiv preprint arXiv:1707.08817*, 2017.
- [20] A. Rajeswaran, V. Kumar, A. Gupta, G. Vezzani, J. Schulman, E. Todorov, and S. Levine, “Learning complex dexterous manipulation with deep reinforcement learning and demonstrations,” in *Proceedings of Robotics: Science and Systems*, 2018.
- [21] A. Nair, A. Gupta, M. Dalal, and S. Levine, “AWAC: Accelerating online reinforcement learning with offline datasets,” *arXiv preprint arXiv:2006.09359*, 2020.
- [22] P. J. Ball, L. Smith, I. Kostrikov, and S. Levine, “Efficient online reinforcement learning with offline data,” in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 1577–1594.
- [23] I. Kostrikov, A. Nair, and S. Levine, “Offline reinforcement learning with implicit q-learning,” *arXiv preprint arXiv:2110.06169*, 2021.
- [24] C. Shen and C. Sloth, “Solving robotics tasks with prior demonstration via exploration-efficient deep reinforcement learning,” *Frontiers in Robotics and AI*, vol. 12, p. 1682200, 2025.
- [25] B. Lakshminarayanan, A. Pritzel, and C. Blundell, “Simple and scalable predictive uncertainty estimation using deep ensembles,” *Adv. Neural Inf. Process. Syst.*, vol. 30, 2017.
- [26] S. Fujimoto and S. S. Gu, “A minimalist approach to offline reinforcement learning,” *Adv. Neural Inf. Process. Syst.*, vol. 34, pp. 20 132–20 145, 2021.
- [27] A. P. Lindvig, I. Iturrate, U. Kindler, and C. Sloth, “ur_rtde: An Interface for Controlling Universal Robots (ur) Using the Real-time Data Exchange (rtde),” in *2025 IEEE/SICE International Symposium on System Integration (SII)*. IEEE, 2025, pp. 1118–1123.
- [28] A. Kumar, A. Zhou, G. Tucker, and S. Levine, “Conservative q-learning for offline reinforcement learning,” *Adv. Neural Inf. Process. Syst.*, vol. 33, pp. 1179–1191, 2020.